**1. Introduction**

Graph Traversal is the process of visiting all the vertices (nodes) of a graph in a systematic manner. The two main graph traversal algorithms are:

1. Breadth-First Search (BFS)
2. Depth-First Search (DFS)

Both algorithms use a visited array or set to avoid visiting the same vertex repeatedly.

**2. Breadth-First Search (BFS)**

Definition: Breadth-First Search is a graph traversal algorithm that visits vertices level by level, exploring all neighbouring vertices before moving to the next level.

Data structure used: Queue (FIFO — First In, First Out).

**Algorithm**

1. Start from the source vertex.
2. Mark it as visited and insert it into the queue.
3. Remove the front vertex from the queue and process it.
4. Visit all its unvisited neighbours and add them to the queue.
5. Repeat until the queue becomes empty.

**Example**

Suppose the graph has edges A–B, A–C, B–D and C–E.

BFS traversal starting from A:

A → B → C → D → E

**3. Depth-First Search (DFS)**

Definition: Depth-First Search is a graph traversal algorithm that explores one branch as deeply as possible before backtracking to explore another branch.

Data structure used: Stack (LIFO — Last In, First Out) or recursion.

**Algorithm**

1. Start from the source vertex.
2. Mark the vertex as visited.
3. Visit one unvisited neighbour and continue deeper.
4. If no unvisited neighbour remains, backtrack.
5. Continue until all reachable vertices have been visited.

**Example**

For the same graph, one possible DFS traversal is:

A → B → D → C → E

Note: The traversal order can vary depending on the order in which neighbours are selected.

**4. Difference Between BFS and DFS**

| BFS | DFS |
|---|---|
| Visits nodes level by level. | Explores deeply before backtracking. |
| Uses a queue. | Uses a stack or recursion. |
| Finds shortest paths in unweighted graphs. | Does not guarantee shortest paths. |
| Useful for level-order exploration. | Useful for cycle detection and path exploration. |

**5. Time and Space Complexity**

Let \(V\) be the number of vertices and \(E\) be the number of edges.

- Time Complexity: \(O(V+E)\) using an adjacency list.
- Space Complexity: \(O(V)\) for the visited structure and queue, stack, or recursion stack.

**6. Applications**

BFS:

- Shortest path in an unweighted graph.
- Social network connections.
- Level-order traversal.

DFS:

- Cycle detection.
- Connected components.
- Topological sorting.
- Maze solving and path exploration.